Sheepy

Sleep planning with a physical companion

Sheepy turns tomorrow's commitments and last night's sleep into a small, readable plan: when to stop caffeine, start winding down, wake up, and get morning light. A connected sheep brings part of that routine off the screen, pairing a gradual light fade with a familiar gesture: a pat.

The problem we chose

A sleep chart describes the night that already happened. Tomorrow's calendar tells a different story. Our prototype connects those two views so a person can see what to do next, why the recommendation changed, and how an early commitment affects the evening before.

A dashboard with a purpose

The iPhone experience uses a quiet indigo and violet palette, a sheep figure inspired by the physical enclosure, and a timeline directly beneath it. Demo and live inputs are clearly distinguished. Calendar and Apple Health badges appear when those sources have actually supplied data.

The "Why this plan?" sheet explains the exact result already on screen. It works locally, without an API key or network connection. A poor-night demo moves the relevant evening recommendations earlier; resetting restores the normal fixture.

A small physical ritual

The morning-light demo fades the connected lamp toward 80% brightness over 12 seconds. A new pat tells the app to send an immediate light-off command. The touch remains in the pat log, giving the interaction a visible record rather than making it disappear into a button press.

Design intent. Make the next step easy to understand and satisfying to carry out. The physical sheep is an interface for the routine, not a claim that touch or light readings can measure sleep quality.

iPhone app • Local planning • Bluetooth hardware • Offline plan explanations

A plan you can inspect

The planner is deterministic: the same inputs produce the same times. It does not need a language model to calculate a schedule, and it retains the facts used to explain each result.

Worked example

An illustrative local-day scenario uses a preferred wake time of 7:00 AM, 60 minutes to prepare, an eight-hour sleep target, and a 7:00 AM commitment tomorrow. All times below use America/Los_Angeles. “Today” is the evening before the wake day.

Recommendation	Normal demo	Poor-night demo
Stop caffeine	Today · 2:00 PM	Today · 1:30 PM
Start winding down	Today · 9:00 PM	Today · 8:30 PM
Wake up	Tomorrow · 6:00 AM	Tomorrow · 6:00 AM
Get morning light	Tomorrow · 6:30–7:00 AM	Tomorrow · 6:30–7:00 AM

What changes and why

The early commitment moves waking to 6:00 AM so preparation still fits. The normal fixture contains eight hours of sleep; the poor-night fixture contains five and a half. Below six hours, the prototype adds 30 minutes of sleep opportunity by moving bedtime and the evening recommendations earlier. Wake time remains anchored to the commitment.

The rules behind the four times

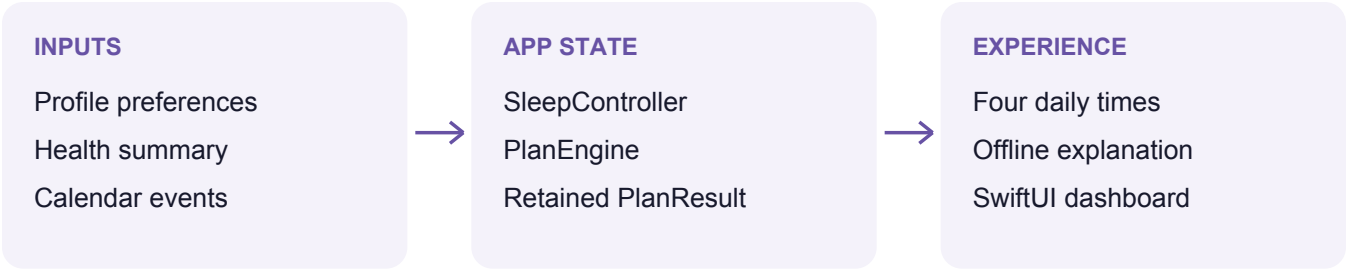
Timed commitments beginning before 9:00 AM can move waking earlier than the preferred time. All-day events do not anchor waking. Wind-down begins one hour before planned bedtime; caffeine cutoff is eight hours before it. The morning-light window begins 30 minutes after waking and lasts 30 minutes.

Dates and explanations stay together

The controller calculates tomorrow in the profile’s time zone and calls **PlanEngine.makeResult** once per generation. The timeline displays its plan; **OfflineExplainer** uses that same retained result. Input changes regenerate both, and missing or invalid results do not leave an old explanation on screen.

Scope. These are transparent prototype rules, not medically validated recommendations. The demo illustrates adaptation; it does not establish a health outcome.

From data to a physical response



One owner of app state

SleepController coordinates the existing SwiftUI dashboard, providers and hardware connection. Calendar and Health requests have independent loading and error tracking. Request tokens prevent old responses from replacing current state after a mode change. Hardware simulation is separate from demo versus live planning data.

Health and Calendar at the boundary

EventKit supplies calendar events after an explicit access request. HealthKit requests read access to sleep analysis and heart rate, then summarizes a completed overnight session locally. Sleep intervals are kept to a consistent source and overlaps are merged before duration is counted. Missing heart rate does not discard sleep.

A completed Health authorization flow is not treated as proof of read access. Empty results remain an honest empty state. Raw health samples are not added to chat requests.

Bluetooth is the hardware connection

The iPhone communicates directly with the Arduino over Bluetooth Low Energy. The Python USB bridge remains useful for the earlier laptop setup, but is not required for the phone. Status notifications carry pat count, relative room brightness, current lamp level and target level. Fade commands specify brightness and duration.

The Arduino performs the fade; the app observes new pats and sends light-off. The demo serializes a pat-triggered off command behind an outstanding fade write, avoiding overlapping commands. Manual controls, chat, disconnection or changing hardware mode disarm the demo.

Two distinct explanation paths

Plan explanations are entirely offline. A separate optional Claude chat can operate the lamp through existing tools and requires network access and configuration. Chat is not required to calculate the plan, explain it, or run the manual hardware demo.

Hardware boundary. The team reports five wired LEDs. The checked-in firmware lists two output pins; all-five-LED behavior depends on the uploaded firmware and wiring. Room brightness is a relative sensor reading, not lux.

Show the experience in two minutes

1 · Establish the plan. Open the normal demo. Point out tomorrow's 7:00 AM commitment, the 6:00 AM wake recommendation, and today's caffeine and wind-down times.

2 · Make the change visible. Tap "I slept poorly." Show the earlier evening recommendations, open "Why this plan?", then reset. Explain that the text comes from the same result as the timeline.

3 · Bring in the sheep. In Lamp & chat, turn hardware simulation off and connect. Disconnect nRF Connect first. Cover and uncover the light sensor to show the relative reading, then start the morning light demo.

4 · Finish with a pat. During or after the 12-second fade, pat Sheepy. The app should command the light off and retain the pat in its log. Keep the app open and connected. A simulated wake-up pat is available for rehearsal without hardware.

What has been demonstrated or checked

Area	Evidence at this handoff
Phone and board	Team-confirmed Bluetooth connection, manual light control and pat logging.
Planning and explanations	59 planner checks and 18 offline-explainer checks passed in integration.
App integration	Full simulator build and controller checks passed, including normal → poor night → reset.
Morning light demo	Automated checks passed for pat-to-off, pats during pending writes, duplicate pats, off failures and reconnect safety. Physical walkthrough still pending.

What comes next

First, verify the full fade-and-pat loop on the phone and confirm all five LED outputs. Next, add timestamped events buffered on the sheep so touches can survive a disconnected phone. Intentional bedtime and wake gestures can then become editable session records, clearly labeled as time between gestures rather than measured sleep.

Reliable scheduled wake lighting, background operation and sensor-based adaptation remain future work. The current demo starts immediately; it does not schedule an alarm or infer sleep from pats. Health and Calendar permission flows still need device-level verification.

Project reference

github.com/Aesha7/Sheepy · person-a-integration · 0b01cec
Technical basis: PlanEngine, OfflineExplainer, SleepController, HealthKitService, BluetoothSheep and the drift-agent hardware firmware. Validation statements reflect the integration record and team-reported device checks.